INTRODUCTION

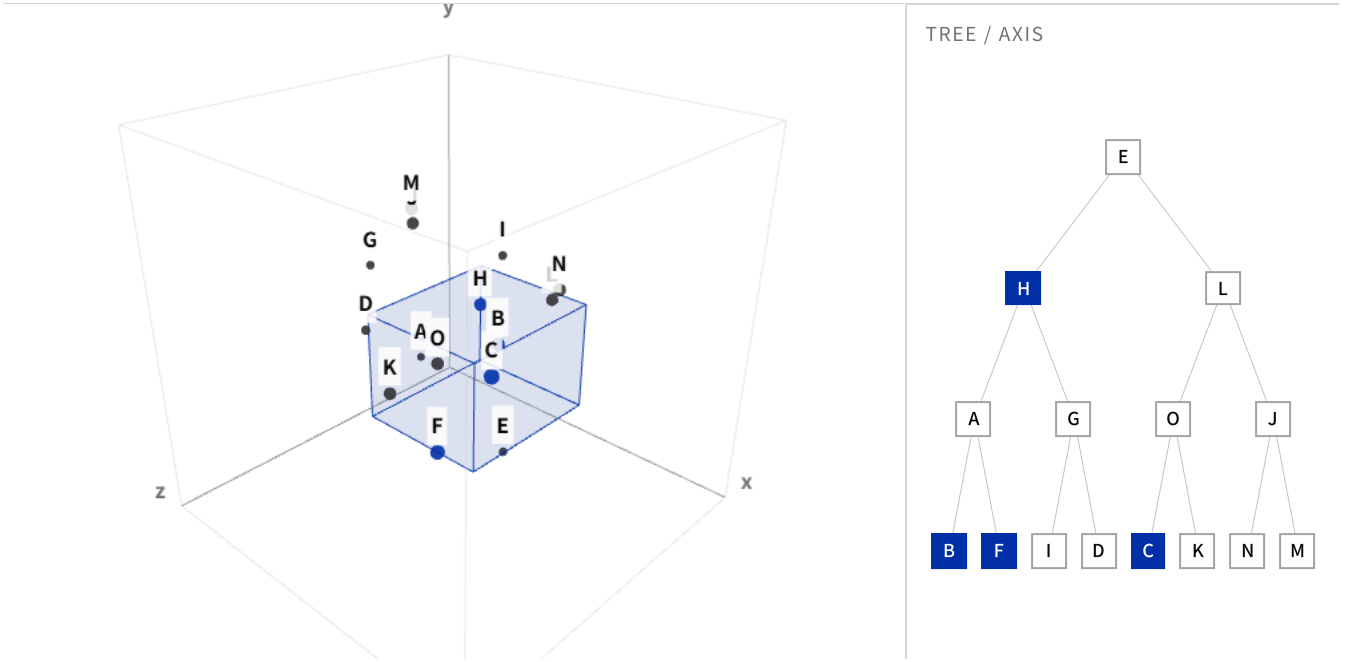
KD-tree：多维空间查询

把 N 个点组织成层次，让查询只进入仍可能产生答案的空间。

划分空间

减少访问

三维点云负责建立直觉；定义、正确性与复杂度面向一般 *d* 维。



三维点云 / 长方体范围查询 · 拖动旋转

固定维度 D · 点数 N · 输出 R

MOTIVATION

为什么不能逐点扫描?

WITHOUT INDEX

N

每次查询都检查全部点。

WITH SPATIAL STRUCTURE

$V \ll N$

预处理后，只访问仍可能相关的区域。

> 比较访问的节点数：扫描 N 个，索引访问 V 个。逐维判断需 O(d)，范围报告还需输出成本；最坏 V 仍可达到 N。

CONSTRUCTION

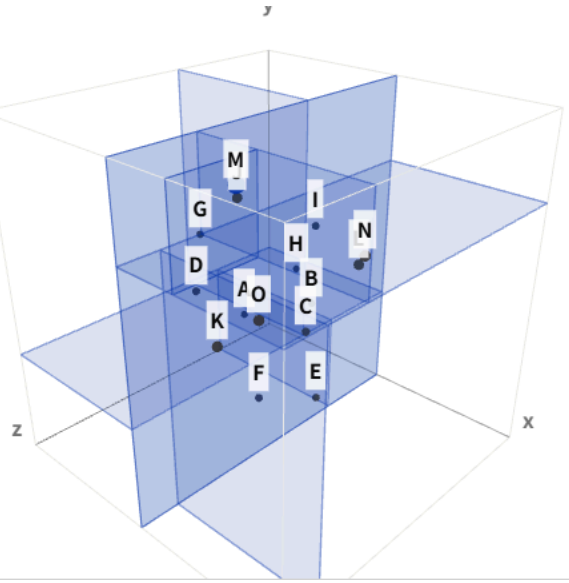
一层切分，一类约束

axis = depth mod *d*

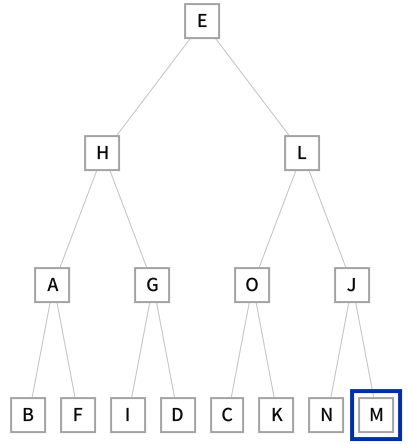
排序选中维 → 取中位数 → 左右递归

划分区域
祖先切分约束累积出的空间。

紧包围盒
子树实际点的逐维最小/最大值。



TREE / AXIS



选取中位数并划分 · M · x 轴 · 深度 3

CASE STUDY

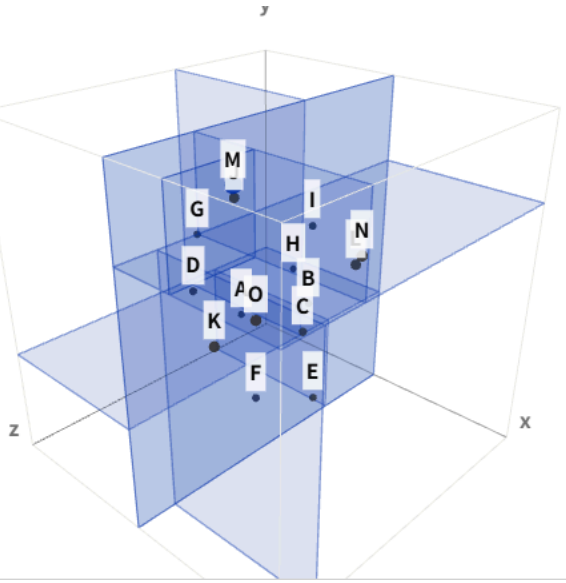
三维点集建树

15

个唯一编号点，循环执行 x → y → z → ⋯

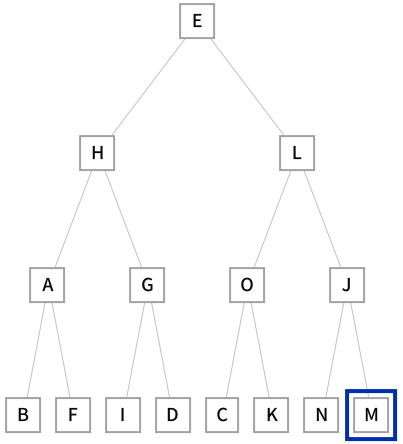
层	节点	轴
0	E (8, 1, 5)	x
1	H · L	y
2	继续递归	z

中位数取 len(points) // 2；本案例根节点左右各 7 点。



选取中位数并划分 · M · x 轴 · 深度 3

TREE / AXIS



IMPLEMENTATION

实现与构建代价

PYTHON / BUILD

```python
def build(points, depth=0):
    if not points:
        return None
    axis = depth % d
    ordered = sorted(points,
        key=lambda p: (p.coords[axis], p.id))
    mid = len(ordered) // 2
    left = build(ordered[:mid], depth + 1)
    right = build(ordered[mid + 1:], depth + 1)
    return make_node(ordered[mid], axis, left, right)
```

上述排序/选择按单坐标比较计费；自底向上合并 d 维包围盒另需 O(dN)。空间 O(dN)。

01 / 每层排序工作量

第 ℓ 层约有 2^{ℓ} 个子问题，每个规模 $N/2^{\ell}$。

$$2^{\ell} \cdot O((N/2^{\ell}) \log(N/2^{\ell}))$$
$$= O(N(\log N - \ell))$$

02 / 累加 $L = \lceil \log_2 N \rceil$ 层

$$B(N) = O(N \sum_{\ell=0}^{L-1} (L-\ell))$$
$$= O(N \cdot L(L+1)/2)$$
$$= O(N \log^2 N)$$

03 / 若改用线性中位数选择 每层合计 O(N)，共 O(log N) 层，得到 O(N log N)。

RANGE QUERY

范围查询的三种关系

PYTHON / REPORT

```python
def report(node, lo, hi, out):
    if node is None:
        return
    state = relation(node, lo, hi)
    if state == "disjoint":
        return
    if state == "contained":
        out.extend(subtree_points(node))
        return
    if inside(node.point, lo, hi):
        out.append(node.point)
    report(node.left, lo, hi, out)
    report(node.right, lo, hi, out)
```

$$Q = \prod_{i=1}^{d} [a_i, b_i]$$

不相交 / DISJOINT

整棵子树跳过，不检查其内部点。

完全包含 / CONTAINED

直接收集子树全部点。报告必须支付 O(r) 输出成本。

部分相交 / PARTIAL

判断当前点，再递归左右孩子。

调用：out = []; report(root, lo, hi, out)

relation 做 d 维包围盒比较；inside 按闭区间判断。subtree_points 以 O(m) 前序收集，不排序。

RANGE DEMO

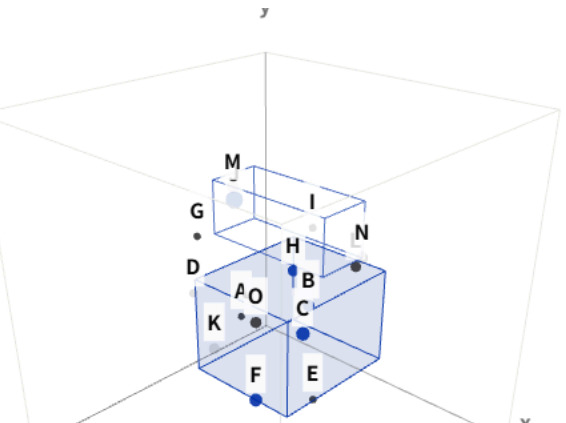
三维范围查询回放

$Q = [3,9] \times [2,7] \times [1,8]$

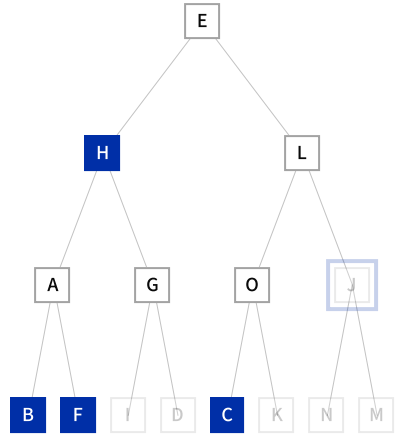
4

命中 B、C、F、H

VISIT · PRUNE · ACCEPT · HIT



TREE / AXIS



不相交 · 跳过子树 · J · 访问 13 节点 · 已命中 4 点

回放同时显示树上节点与累计访问数；“接收子树”和“单点命中”用文字标签区分。

CORRECTNESS

范围计数与正确性

PYTHON / COUNT

```python
def count(node, lo, hi):
    if node is None:
        return 0
    state = relation(node, lo, hi)
    if state == "disjoint":
        return 0
    if state == "contained":
        return node.size
    hit = int(inside(node.point, lo, hi))
    return (hit + count(node.left, lo, hi)
            + count(node.right, lo, hi))
```

与范围报告唯一关键差别

完全包含 → 返回 node.size

不用收集每个点，因此计数没有 O(r) 输出项。

> **不变式**
> 子树点全部在 B 中，size 等于子树点数。

不相交分支不漏解；包含分支不增错；其余分支由当前点和两棵子树完整覆盖。按树归纳得正确性。

调用 count(root, lo, hi) 返回整数。计数与报告共用相同包围盒、闭区间与点包含判定。

DERIVATION

一般 d 维复杂度推导

固定 D ⩾ 2 · 静态中位数平衡 · 循环选轴 · 轴对齐查询盒 · 一般位置

01 / 数边界穿过的区域

取查询盒的一张边界超平面。完整 d 层中，法向轴只进入一侧，其余 d−1 轴最多各进入两侧。

子问题数 $a = 1 \cdot 2^{d-1}$

规模缩小倍数 $b = 2^d$

02 / 写出递推

$$T_d(N) \leqslant a\,T_d(N/b) + C_d$$

C_d 是固定 d 层内的工作，可依赖维度；$T_d(O(1)) = O_d(1)$。

> d=3：a=4、b=8。每轮最多 4 个子问题，规模各缩至约 1/8。

先分析一张边界；查询盒有 2d 个面，固定 d 时只增加常数因子。下一页再计入输出成本。

03 / 展开 J 轮，不直接跳到结果

$$T_d(N) \leqslant a^j T_d(N/b^j) + C_d \sum_{i=0}^{j-1} a^i$$

04 / 展开到常数规模

取 $L \approx \log_b N$。因 $a > 1$，几何级数与叶子项均为 $O_d(a^L)$。

$$a^L = N^{\log_b a}$$

$$\log_{2^d} 2^{d-1} = (d-1)/d$$

$$T_d(N) = O_d(N^{1-1/d})$$

COMPLEXITY

报告、计数与维度

报告：$O_d(N^{1-1/d} + r)$

计数：$O_d(N^{1-1/d})$

维度	报告	计数
d = 2	$O(\sqrt{N} + r)$	$O(\sqrt{N})$
d = 3	$O(N^{2/3} + r)$	$O(N^{2/3})$
固定 d ⩾ 2	$O_d(N^{1-1/d} + r)$	$O_d(N^{1-1/d})$

为何加 r?

边界区域继续展开；完整包含的子树停止几何判断。报告仍需逐点输出，共 O(r)；计数直接加 size，无输出项。

为何单独处理 d=1?

此时 a=1，展开式的和为 L 而非几何增长，恢复 O(log N)；高维时指数趋近 1，剪枝可能减弱。

NEAREST NEIGHBOR

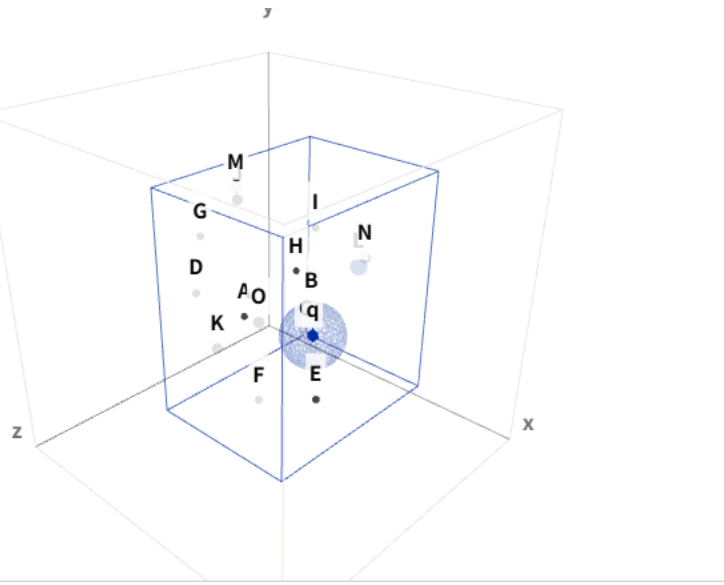
最近邻的距离下界

$$\delta_i = \max(l_i - q_i, 0, q_i - u_i)$$

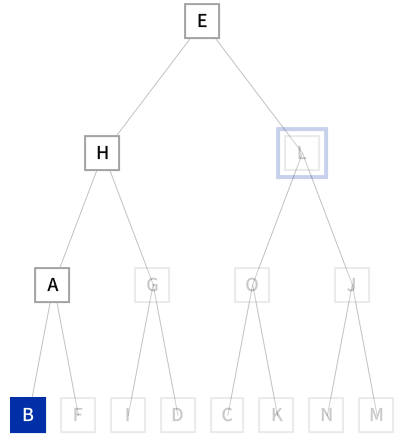
$$\mathrm{dist}_{\min}^2(q, B) = \sum_{i=1}^{d} \delta_i^2$$

> **剪枝** $\mathrm{dist}_{\min}^2(q, B) > \mathrm{best}^2$

先访问下界更小的子树。



TREE / AXIS



距离下界过大 · 剪枝 · L · 候选 B · 距离² 3.00

3D 实例：q=(6,3,3) → B=(5,4,2)，距离平方 3。比较平方距离，不用开方。

TRADEOFFS

最近邻代码与查询成本

PYTHON / NEAREST

```python
def nearest(root, q):
    best = (inf, "", None)  # 距离平方、ID、点
    def visit(node):
        nonlocal best
        if node is None or lower_bound(q, node) > best[0]:
            return
        best = min(best, (distance2(q, node.point),
                          node.id, node.point))
        children = [n for n in (node.left, node.right) if n]
        children.sort(key=lambda n: (lower_bound(q, n), n.id))
        for child in children:
            visit(child)
    visit(root)
    return best[2], best[0]
```

候选与等距规则

best 保存（距离平方、ID、点）。先比较距离，再以 ID 打破平局。

严格大于才剪枝

下界等于 best[0] 时继续搜索，以免遗漏更小 ID 的等距点。

先搜索下界小的子树

至多两个孩子，排序项数固定，不引入 log N 因子。

访问 V 个节点 → O(dV)

V ⩽ N ⇒ 最坏 O(dN)

树高限制单条路径，不能限制所有回溯路径的总访问量。

lower_bound 是上一页的包围盒距离下界；distance2 是 d 维欧氏距离平方。空树返回 (None, inf)。

DYNAMIC INSERT

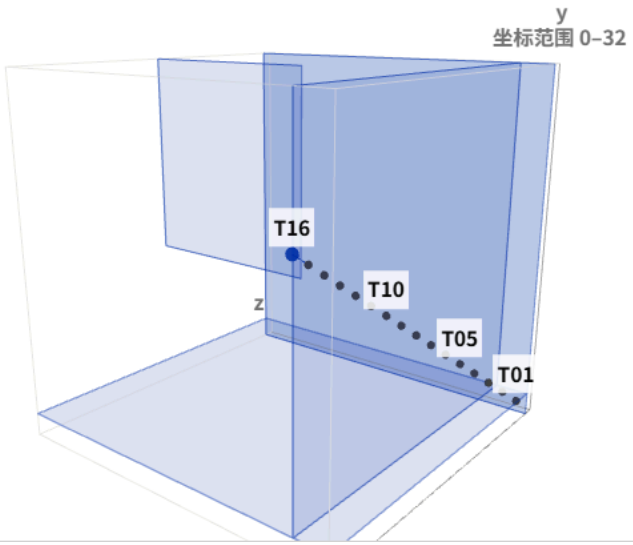
插入如何破坏平衡

比较当前轴 → 递归一侧 → 回溯维护

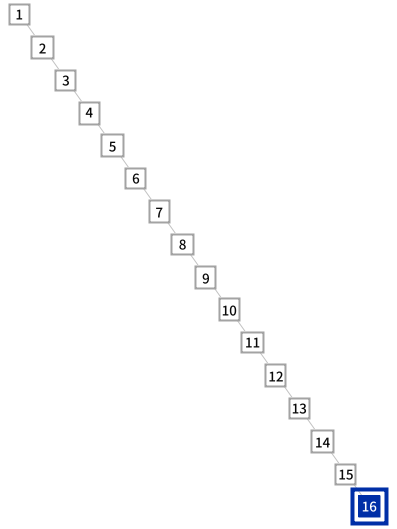
$$p_t = (t, t, \cdots, t)$$

$$h = \Theta(N)$$

每一轴都向右，形成单链。



PLAIN · 无重建



无重建插入 16 点 · h = 16 · 最新点 (16, 16, 16)

> 回溯必须同步更新 size 与 bbox；单调序列只说明退化风险。

REBUILDING

局部重建恢复树高

$$\max(\text{size}(L), \text{size}(R)) > \alpha \cdot \text{size}(u), \quad \alpha \in (1/2, 1)$$

A / 最大孩子占比阈值

允许任一孩子占整棵子树的比例上限。分母 size(u) 包含当前节点自身。

本例 A = 0.75

9/12 = 75%：不触发。

向大孩子再插一点：10/13 ≈ 76.9%，触发。

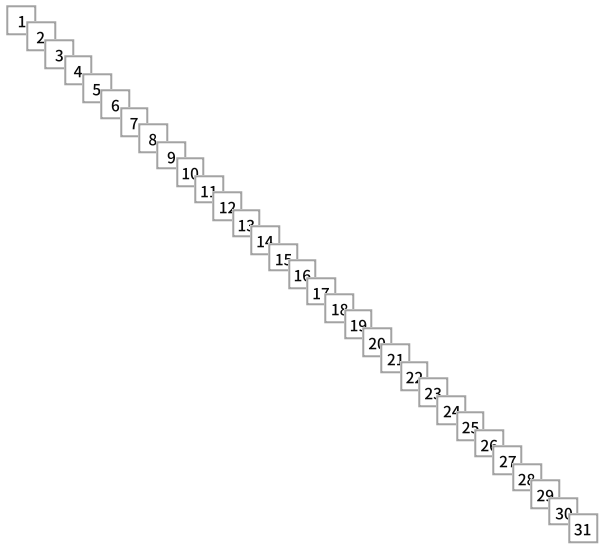
小 A 更严格，大 A 更宽松

α ↓：通常更频繁重建、树更矮。

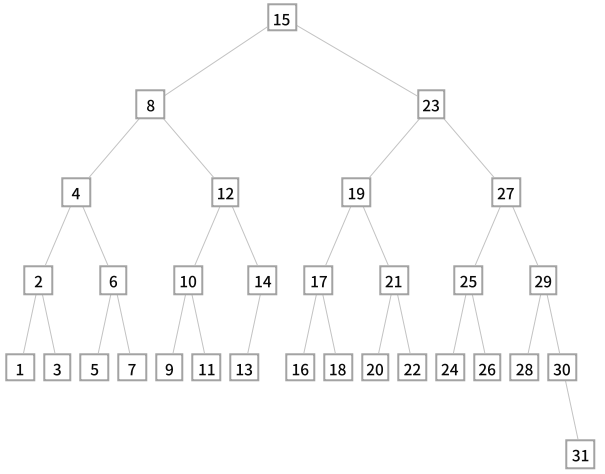
α ↑：容忍更偏的树，路径可能更长。

触发后：收集点 → 中位数重建 → 接回并维护。

PLAIN · 无重建



REBUILD · α = 0.75



已插入 31 点 · 树高 31 → 6 · 9 次重建

固定 1/2 < α < 1：为插入留出余量，同时保证沿路径规模收缩；α 衡量点数比例，不是空间体积。

COST MODEL

重建成本与摊还分析

01 / 单次成本

收集 O(m)；递归排序加包围盒合并：

$$C(m) = O(m \log^2 m + dm)$$

正常路径 O(d log N)；根级重建时，单次最坏上界为 O(N $\log^2$N+dN)。

02 / 树高保证

$$1 \leqslant \alpha^k N \Rightarrow h \leqslant 1+\log_{1/\alpha} N$$

候选以上已平衡且点数不变；以下整体重建，路径外不变，故恢复全树 α 约束。

理论优化版本 若 C(m)=O(m log m+dm)，同法得到摊还 $O_\alpha(\log^2 N+d \log N)$。

03 / 重建之间至少多少插入?

初始规模 s，中位数划分后孩子 ⩽ s/2；新增 t 点，m=s+t。

$$\alpha m < s/2+t = m/2+t/2$$

$$\Rightarrow t > (2\alpha-1)m = \Omega_\alpha(m)$$

04 / 分摊给路径上的更新

每个祖先每次存 $O_\alpha(\log^2 N+d)$ 信用，足够支付下一次 C(m)。每条路径有 $O_\alpha(\log N)$ 个祖先。

$$\text{摊还插入：} O_\alpha(\log^3 N+d \log N)$$

信用绑定节点实例，重建后旧账户作废。仅从空树纯插入、固定 α、关闭追踪；N 为序列最大规模。

单次最坏 ≠ 摊还上界 ≠ 动态查询界。上述摊还界按本次重建策略推导，不直接套用普通替罪羊树的 O(log N)。

EXPERIMENTS

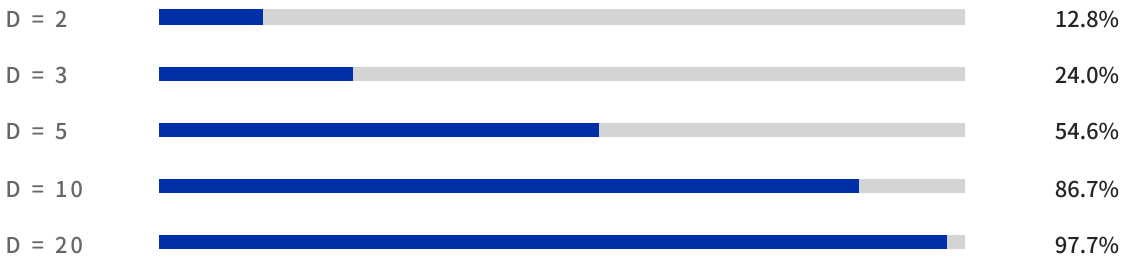
用实验验证什么

范围查询 · d=3

N	KD/ms	暴力/ms	访问占比
250	0.046	0.099	13.2%
500	0.044	0.197	8.1%
1000	0.088	0.428	7.7%
2000	0.138	0.825	5.9%

CORRECTNESS | SCALE · D = 3 | DIMENSION · FIXED N | REBUILD ON / OFF

维度变化 · N=512



访问节点比例；查询盒体积比例固定为 8%，输出数量另见原始记录。

指标	记录
时间	建树 / 查询分开，多轮中位数
访问与输出	访问节点数 / 访问比例 / 输出点数 r
动态	树高 / 重建次数 / 重建规模

Python 实测 · 固定随机种子 · 每批查询运行 3 轮取中位数 · 动画与事件追踪不计入耗时。

APPLICATION

应用与结构选择

- **点云** 区域裁剪与邻域候选
- **空间检索** 轴对齐区域筛选
- **机器学习** 适合低维的近邻搜索

选择前核对

维度 × 分布 × 查询形状 × 更新频率 × 输出规模 × 存储预算

开头场景可以闭环：范围查询取工作区域，最近邻找最近采样点，持续采样时局部重建维持树高。

SUMMARY

总结：结构与查询

01 STRUCTURE

循环选轴 × 中位数划分

02 QUERY

包含关系 × 距离下界

$$R(N, r) = O_d(N^{1-1/d} + r)$$

$$C(N) = O_d(N^{1-1/d})$$

固定 d≥2 · 经典静态条件

> **动态维护** 局部重建保证树高，不自动保证查询或摊还更新界。

[1] Sedgewick & Wayne · Geometric Search [2] Morin · ScapegoatTree / partial rebuilding